

CastagNet

Un logiciel qui reconnaît la qualité des châtaignes à partir de photos

Mise en Situation Professionnelle — MSc BIHAR / ESTIA

Auteur : **Mario Caballero** · Code : github.com/ayala2a/castagnet

Comment j'ai travaillé. Je me suis aidé d'une intelligence artificielle (Claude) comme assistant, pour analyser les données, écrire le code et corriger les erreurs. Les choix et la direction du projet restent les miens.

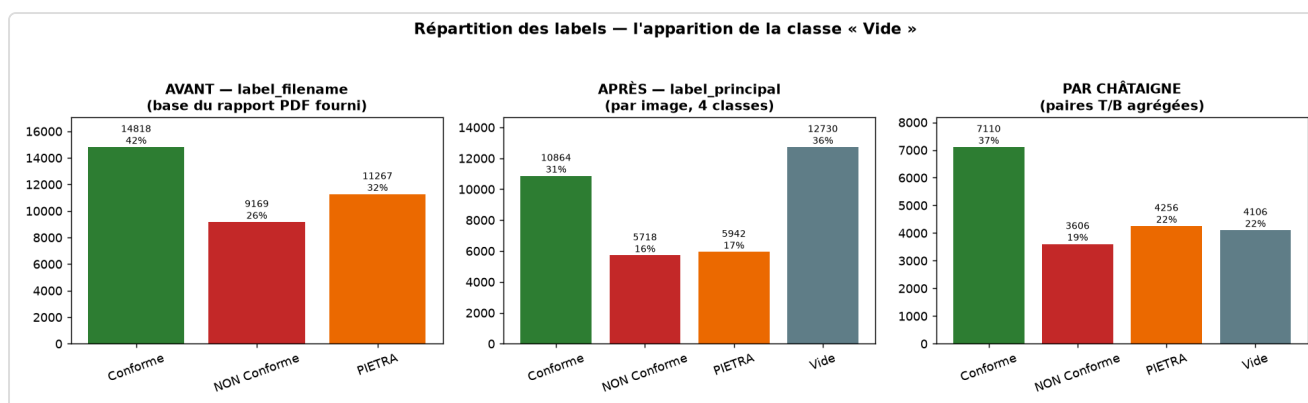
1. Le problème

Une coopérative corse trie ses châtaignes sèches sur une machine automatique qui photographie chaque fruit du dessus et du dessous. Il faut décider s'il est **bon**, **mauvais**, d'une variété à écarter (**PIETRA**), ou si l'emplacement est **vide**. La règle de la coopérative guide tout : **mieux vaut jeter un bon fruit que laisser passer un mauvais** (un mauvais fruit gâche la farine). Concrètement : parmi les fruits déclarés « bons », au moins 95 % doivent l'être vraiment, et il faut retrouver au moins 85 % des bons — en direct, sur un ordinateur modeste.

Deux mots à retenir. **Précision** = « quand le logiciel dit bon, a-t-il raison ? ». **Rappel** = « retrouve-t-il bien tous les bons fruits ? ».

2. Le travail sur les données

En vérifiant les 35 254 photos, j'ai trouvé que le rapport fourni était faux : il oubliait les emplacements **vides**, qui représentent en réalité **36 % des photos**. Je l'ai refait correctement, et terminé la vérification d'un lot mal étiqueté.



À gauche l'ancien rapport (incomplet), à droite la réalité avec les emplacements vides.

Une châtaigne = deux photos. J'ai relié la vue du dessus et celle du dessous de chaque fruit. Découverte importante : quand les deux photos ne sont pas d'accord, c'est toujours parce que l'une ne voit pas le fruit — jamais un vrai conflit de qualité. Donc **on ne peut pas se fier à une seule photo** (un fruit sur trois paraît « vide » sur une de ses vues), il faut regarder les deux.

Les vidéos. Deux vidéos filment le même fruit du dessus et du dessous, mais légèrement décalées dans le temps. Plutôt que d'aligner « à l'œil », j'ai mesuré le décalage en repérant les moments où les fruits passent : c'est un décalage **net de 5 images (0,2 s)**. J'ai ainsi relié les fruits d'une vidéo à l'autre.



Fruits reliés depuis les vidéos : chaque colonne est le même fruit, vu du dessus (haut) et du dessous (bas).

Enfin, pour l'entraînement, je répartis les fruits en trois groupes (apprendre / régler / tester) en gardant **toujours les deux photos d'un fruit dans le même groupe** — sinon le logiciel pourrait « reconnaître » un fruit déjà vu et tricher sans le vouloir.

3. Le logiciel et ses résultats

J'ai construit plusieurs versions, de plus en plus fines, guidé par les résultats :

- **Une version simple** (une seule photo) : elle rate presque tous les PIETRA, car le défaut ne se voit souvent que d'un côté.
- **Deux photos ensemble**, avec un modèle léger et rapide (adapté à la machine modeste).
- **+ la différence entre les deux photos** : si un défaut n'est visible que d'un côté, les deux photos se ressemblent moins à cet endroit. Cette idée a débloqué le rappel.

- **Version « radiale »** : je déroule le disque de l'image à plat avant de l'analyser. Peu importe comment le fruit est tourné, ça revient au même. C'est ma **meilleure version**.

Pour décider, le logiciel ne déclare « bon » que s'il en est **vraiment sûr** — c'est ce réglage qui garantit la précision de 95 % demandée.

Version	Bonnes réponses	Précision / rappel sur « bon »	Objectif
Version simple (1 photo)	66 %	95 % / 30 %	non
Deux photos ensemble	86 %	96 % / 77 %	non
+ différence entre les photos	91 %	95 % / 88 %	oui
Version radiale (retenue)	92 %	95 % / 92 %	oui, avec marge

Les erreurs qui restent sont des confusions « bon / PIETRA » (défauts très fins), et surtout elles **vont dans le bon sens** : le logiciel écarte parfois un bon fruit par prudence, mais laisse très rarement passer un mauvais. Pour être sûr que ce n'est pas un coup de chance, j'ai **refait l'expérience 5 fois** : les résultats restent stables.

4. Sur la vraie machine

J'ai converti le logiciel dans un format standard et léger, et vérifié qu'il donne les mêmes réponses. Il pèse ~15 Mo, utilise peu de mémoire (~220 Mo en marche, pour 3 Go disponibles) et répond en ~0,05 seconde par cycle. **Toutes les exigences sont respectées**, avec une marge confortable. À l'usage, on lui donne les deux photos d'un fruit et il renvoie sa catégorie.

5. Conclusion

Le logiciel final **respecte la consigne avec de la marge** (95 % de précision, 92 % de rappel sur des fruits jamais vus), tient largement sur la machine, et son résultat est prouvé stable. L'essentiel n'a pas été d'empiler des calculs, mais d'avoir **bien compris le problème** : une châtaigne se juge sur ses deux faces. En gardant toujours la règle « la sûreté avant tout », j'ai évité de courir après un chiffre qui n'aurait pas tenu en vrai.

Outils utilisés : Python, PyTorch, OpenCV, MLflow, ONNX, DVC, Git. Détails techniques dans l'annexe du dépôt.